

Nguyễn Trung Anh - HE190843

IA2001

FPT University

Bài Lab 15

Listing Modules with lm

lm giúp liệt kê tất cả các module (file .exe, .dll) đang chạy trong hệ thống, hiển thị địa chỉ bộ nhớ của chúng. Chữ (deferred) có nghĩa là WinDbg chưa tải thông tin chi tiết cho module đó để tiết kiệm tài nguyên. Nó giúp có cái nhìn tổng quan về những gì đang hoạt động trong kernel.

```
Command - Dump C:\Windows\livekd.dmp - WinDbg:10.0.19041.5609 AMD64
For analysis of this file, run !analyze -v
0: kd> lm
start      end                                module_name
00007fff7`3c9b0000 00007fff7`3ca2e000  vmacthlp (deferred)
00007ffa`0a910000 00007ffa`0a922000  csccapi (deferred)
00007ffa`0ba80000 00007ffa`0bab5000  winbrand (deferred)
00007ffa`0bac0000 00007ffa`0bb1b000  framedynos (deferred)
00007ffa`0bb20000 00007ffa`0bd2c000  ciavini32 (deferred)
00007ffa`0eb40000 00007ffa`0eb58000  NCObiAPI (deferred)
00007ffa`0f7a0000 00007ffa`0f7af000  perios (deferred)
00007ffa`101d0000 00007ffa`102dc000  FastProx (deferred)
00007ffa`10600000 00007ffa`10628000  vniutils (deferred)
00007ffa`10750000 00007ffa`10764000  wbeasvc (deferred)
00007ffa`10c20000 00007ffa`10cb0000  wbeascom (deferred)
00007ffa`10d40000 00007ffa`10d4a000  rasadhlp (deferred)
00007ffa`13420000 00007ffa`1343d000  dhcpcsvc (deferred)
00007ffa`13540000 00007ffa`13557000  dhcpcsvc6 (deferred)
00007ffa`15eb0000 00007ffa`15ec4000  wtsapi32 (deferred)
00007ffa`17cb0000 00007ffa`17cc2000  kernel.appcore (deferred)
00007ffa`18d20000 00007ffa`18db7000  schannel (deferred)
00007ffa`18f30000 00007ffa`18f49000  WKSCLI (deferred)
00007ffa`191d0000 00007ffa`1920b000  ITPHPAPI (deferred)
00007ffa`19210000 00007ffa`192da000  DNSAPI (deferred)
00007ffa`194f0000 00007ffa`1955a000  aswsock (deferred)
00007ffa`19af0000 00007ffa`19b23000  DEVORJ (deferred)
00007ffa`19c10000 00007ffa`19c73000  WINSTA (deferred)
00007ffa`19c80000 00007ffa`19c92000  UMFDIC (deferred)
00007ffa`19ca0000 00007ffa`19ceb000  powrprof (deferred)
00007ffa`19d20000 00007ffa`19d52000  Spicli (deferred)
00007ffa`19e40000 00007ffa`19e62000  win32u (deferred)
00007ffa`19e70000 00007ffa`19f0d000  asvcpc.win (deferred)
00007ffa`19f10000 00007ffa`19f37000  bcrvpt (deferred)
00007ffa`19f40000 00007ffa`19fc2000  bcrvptPrimitives (deferred)
00007ffa`1a130000 00007ffa`1a426000  KERNELBASE (deferred)
00007ffa`1a4e0000 00007ffa`1a5f9000  api32full (deferred)
00007ffa`1a600000 00007ffa`1a64e000  cimgmt32 (deferred)
00007ffa`1a6d0000 00007ffa`1a7d0000  ucrtbase (deferred)
00007ffa`1ab40000 00007ffa`1ab48000  NSI (deferred)
00007ffa`1afd0000 00007ffa`1b06e000  asvcrt (deferred)
00007ffa`1b070000 00007ffa`1b132000  KERNEL32 (deferred)
00007ffa`1b8c0000 00007ffa`1ba61000  user32 (deferred)
00007ffa`1ba70000 00007ffa`1ba9b000  GDI32 (deferred)
00007ffa`1bbc0000 00007ffa`1bc71000  advapi32 (deferred)
00007ffa`1bc80000 00007ffa`1bd4d000  OLEAUT32 (deferred)
00007ffa`1bf40000 00007ffa`1c060000  RPCRT4 (deferred)
00007ffa`1c060000 00007ffa`1c3b4000  combase (deferred)
00007ffa`1c3c0000 00007ffa`1c45f000  sechost (deferred)
00007ffa`1c460000 00007ffa`1c4cb000  WS2_32 (deferred)
00007ffa`1c660000 00007ffa`1c709000  clbcatq (deferred)
```

Danh sách này cho biết trong nhân Windows có rất nhiều hàm khác nhau để tạo/mở file, nhưng quan trọng nhất là `nt!NtCreateFile` — đây chính là hàm mà sẽ dùng cho bước phân tích tiếp theo.

00007ffa`1c3e0000	00007ffa`1c45f000	sechost	(deferred)	
00007ffa`1c460000	00007ffa`1c4cb000	US2_32	(deferred)	
00007ffa`1c4e0000	00007ffa`1c709000	clbcatq	(deferred)	
00007ffa`1c7b0000	00007ffa`1c9a8000	ntdll	(pdb symbols)	c:\symbols\ntdll.pdb\180BF1B90AA75697D0EFEA5E5630AC7E1\ntdll.pdb
ffffbb3f`90a00000	ffffbb3f`90cd6000	win32kbase	(deferred)	
ffffbb3f`90ce0000	ffffbb3f`91094000	win32kfull	(deferred)	
ffffbb3f`910a0000	ffffbb3f`910e9000	cdm	(deferred)	
ffffbb3f`91710000	ffffbb3f`917ab000	win32k	(deferred)	
ffff802`6d140000	ffff802`6d168000	wcupdate_AuthenticAMD	(deferred)	
ffff802`6d170000	ffff802`6d176000	hal	(deferred)	
ffff802`6d180000	ffff802`6d18b000	kd	(deferred)	
ffff802`6d190000	ffff802`6d1b9000	ls	(deferred)	
ffff802`6d1c0000	ffff802`6d22e000	CLFS	(deferred)	
ffff802`6d230000	ffff802`6d24a000	PSHED	(deferred)	
ffff802`6d250000	ffff802`6d25b000	BOOTVID	(deferred)	
ffff802`6d260000	ffff802`6d377000	clbcatq	(deferred)	
ffff802`6d380000	ffff802`6d3ed000	FLTMGR	(deferred)	
ffff802`6d3f0000	ffff802`6d41c000	kssecdd	(deferred)	
ffff802`6d420000	ffff802`6d482000	asrtpc	(deferred)	
ffff802`6d490000	ffff802`6d4a0000	cmacext	(deferred)	
ffff802`6d4b0000	ffff802`6d4c1000	workernl	(deferred)	
ffff802`6d4d0000	ffff802`6d4dc000	ntosex	(deferred)	
ffff802`6d4e0000	ffff802`6d4f3000	UDFIDR	(deferred)	
ffff802`6d500000	ffff802`6d50f000	SleepStudyHelper	(deferred)	
ffff802`6d510000	ffff802`6d521000	VpnRecorder	(deferred)	
ffff802`6d530000	ffff802`6d53e000	asseccore	(deferred)	
ffff802`70a00000	ffff802`71a46000	nt	(pdb symbols)	c:\symbols\ntkrnlap.pdb\F57E740B088E5056E8AF0772F1CC5BEB1\ntkrnlap.pdb
ffff802`72a00000	ffff802`72aec000	CI	(deferred)	
ffff802`72af0000	ffff802`72bab000	cnq	(deferred)	
ffff802`72bb0000	ffff802`72c81000	Wdf01000	(deferred)	
ffff802`72c90000	ffff802`72cb6000	acplex	(deferred)	
ffff802`72cc0000	ffff802`72dc0000	AcPi	(deferred)	

Viewing Memory

dd nt

da nt

Hai lệnh này giúp xác minh rằng vùng bộ nhớ tại địa chỉ nt (đầu của nhân Windows) đúng là một file EXE, với chữ ký "MZ" ở đầu, khẳng định rằng nt trở đúng vào kernel của Windows.

```

0: kd> dd nt
ffff802`70a00000 00905a4d 00000003 00000004 0000ffff
ffff802`70a00010 000000b8 00000000 00000040 00000000
ffff802`70a00020 00000000 00000000 00000000 00000000
ffff802`70a00030 00000000 00000000 00000000 00000118
ffff802`70a00040 0eba1f0e cd09b400 4c01b821 685421cd
ffff802`70a00050 70207369 72676f72 63206d61 6f6e6e61
ffff802`70a00060 65622074 6e757220 206e6920 20534f44
ffff802`70a00070 65646f6d 0a0d0d2e 00000024 00000000
0: kd> da nt
ffff802`70a00000 "MZ.."

```

da nt+4c

Lệnh da nt+4c giúp chứng minh rằng nhân Windows (file ntoskrnl.exe) là một file thực thi tiêu chuẩn, có đầy đủ các đặc trưng của file EXE như phần header DOS và thông báo lỗi tương thích ngược. Đây là bước để khẳng định mình đang nhìn đúng vào vùng nhớ của kernel.

```

0: kd> dd nt
fffff802`70a00000 00905a4d 00000003 00000004 0000ffff
fffff802`70a00010 000000b8 00000000 00000040 00000000
fffff802`70a00020 00000000 00000000 00000000 00000000
fffff802`70a00030 00000000 00000000 00000000 00000118
fffff802`70a00040 0eba1f0e cd09b400 4c01b821 685421cd
fffff802`70a00050 70207369 72676f72 63206d61 6f6e6e61
fffff802`70a00060 65622074 6e757220 206e6920 20534f44
fffff802`70a00070 65646f6d 0a0d0d2e 00000024 00000000
0: kd> da nt
fffff802`70a00000 "MZ."
0: kd> da nt+4c
fffff802`70a0004c ".!This program cannot be run in "
fffff802`70a0006c "DOS mode....$"

```

x nt!*

Lệnh này giúp có cái nhìn tổng quan, nhưng vì danh sách quá lớn, nên sẽ dùng các lệnh có bộ lọc (như *CreateFile*) để tìm kiếm thứ cụ thể thay vì xem toàn bộ.

```

fffff802`71189a40 nt!EtwEnableTrace (EtwEnableTrace)
fffff802`71497b70 nt!ExpTimerMapping = <no type information>
fffff802`70cf38b4 nt!PfSnLogPageFault (PfSnLogPageFault)
fffff802`7174aae8 nt!pXdvZwPrepareComplete = <no type information>
fffff802`7130c2d0 nt!PspTerminatePicoProcess (PspTerminatePicoProcess)
fffff802`713d2c90 nt!VerifierIoInitializeIrp (VerifierIoInitializeIrp)
fffff802`7164a63c nt!HalpTimerWatchdogStopCount = <no type information>
fffff802`7174a6d8 nt!pXdvKeDelayExecutionThread = <no type information>
fffff802`70f18880 nt!KiSaveCurrentEtwTraceBuffer (KiSaveCurrentEtwTraceBuffer)
fffff802`70f4b19c nt!MiRemoveEnclavePagesFromDump (MiRemoveEnclavePagesFromDump)
fffff802`713d1a80 nt!VerifierDbgBreakPointWithStatus (VerifierDbgBreakPointWithStatus)
fffff802`70ef821c nt!HvlpMarkHypervisorPagesForHibernation (HvlpMarkHypervisorPagesForHibernation)
fffff802`712b17b0 nt!PnpTraceDeviceRemovalForResetComplete (PnpTraceDeviceRemovalForResetComplete)
fffff802`712bdd00 nt!NtUmsThreadYield (NtUmsThreadYield)
fffff802`712b32b8 nt!PnpRestoreResourcesInternal (PnpRestoreResourcesInternal)
fffff802`713ebea0 nt!ViZwSetInformationEnlistment (ViZwSetInformationEnlistment)
fffff802`70d1afd0 nt!PspExecuteJobFreezeThawCallback (PspExecuteJobFreezeThawCallback)
fffff802`70f1237c nt!KiDynamicProcessorInitialization (KiDynamicProcessorInitialization)
fffff802`7127e450 nt!CmFcpManagerAllocateChangeSubscription (CmFcpManagerAllocateChangeSubscription)

```

x nt!*CreateFile*

Lệnh này lọc ra tất cả hàm liên quan đến tạo file trong nhân

```

0: kd> x nt!*CreateFile*
fffff802`717420a0 nt!pXdvIoCreateFileSpecifyDeviceObjectHint = <no type information>
fffff802`7136fb50 nt!BiCreateFileDeviceElement (BiCreateFileDeviceElement)
fffff802`7100b890 nt!NtCreateFile (NtCreateFile)
fffff802`7100b6d0 nt!IoCreateFileEx (IoCreateFileEx)
fffff802`70dfa510 nt!ZwCreateFile (ZwCreateFile)
fffff802`713d28f0 nt!VerifierIoCreateFileSpecifyDeviceObjectHint (VerifierIoCreateFileSpecifyDe
fffff802`713e8ed0 nt!VfZwCreateFile (VfZwCreateFile)
fffff802`713e76d0 nt!VerifierNtCreateFile (VerifierNtCreateFile)
fffff802`717420a8 nt!pXdvIoCreateFileEx = <no type information>
fffff802`7174a8d8 nt!pXdvZwCreateFile = <no type information>
fffff802`7109f540 nt!IoCreateFileSpecifyDeviceObjectHint (IoCreateFileSpecifyDeviceObjectHint)
fffff802`713d2840 nt!VerifierIoCreateFileEx (VerifierIoCreateFileEx)
fffff802`7174a6b0 nt!pXdvNtCreateFile = <no type information>
fffff802`7100b920 nt!IopCreateFile (IopCreateFile)
fffff802`7100b630 nt!IoCreateFile (IoCreateFile)
fffff802`7174aec8 nt!pXdvIoCreateFile = <no type information>
fffff802`713b6ae0 nt!KdpCreateFileCallback (KdpCreateFileCallback)
fffff802`713d2790 nt!VerifierIoCreateFile (VerifierIoCreateFile)

```

Unassembling a Function

u nt!NtCreateFile

Lệnh này giúp bạn xem mã Assembly của NtCreateFile để hiểu cách nó hoạt động ở cấp độ thấp, phục vụ cho việc phân tích sâu hơn.

```

0: kd> u nt!NtCreateFile
nt!NtCreateFile:
fffff802`7100b890 4881ec8800000000 sub     rsp,88h
fffff802`7100b897 33c0                                xor     eax,eax
fffff802`7100b899 4889442478                mov     qword ptr [rsp+78h],rax
fffff802`7100b89e c744247020000000 mov     dword ptr [rsp+70h],20h
fffff802`7100b8a6 89442468                mov     dword ptr [rsp+68h],eax
fffff802`7100b8aa 4889442460                mov     qword ptr [rsp+60h],rax
fffff802`7100b8af 89442458                mov     dword ptr [rsp+58h],eax
fffff802`7100b8b3 8b8424e000000000 mov     eax,dword ptr [rsp+0E0h]

```

nt!NtCreateFile

Disassembly Window là một công cụ trực quan giúp bạn thấy rõ rằng hàm NtCreateFile chỉ đóng vai trò là một "cửa ngõ" trung gian, nó sắp xếp dữ liệu và chuyển lời gọi đến `IoCreateFile` - nơi thực sự xử lý việc tạo file

```

Disassembly - Dump C:\Windows\livekd.dmp - WinDbg:10.0.19041.5609 AMD64
Offset: nt!NtCreateFile
fffff802`7100b873 e8a8000000 call nt!IopCreateFile (fffff802`7100b920)
fffff802`7100b878 4881c488000000 add rsp, 88h
fffff802`7100b87f c3 ret
fffff802`7100b880 cc int 3
fffff802`7100b881 cc int 3
fffff802`7100b882 cc int 3
fffff802`7100b883 cc int 3
fffff802`7100b884 cc int 3
fffff802`7100b885 cc int 3
fffff802`7100b886 cc int 3
fffff802`7100b887 cc int 3
fffff802`7100b888 cc int 3
fffff802`7100b889 cc int 3
fffff802`7100b88a cc int 3
fffff802`7100b88b cc int 3
fffff802`7100b88c cc int 3
fffff802`7100b88d cc int 3
fffff802`7100b88e cc int 3
fffff802`7100b88f cc int 3
nt!NtCreateFile:
fffff802`7100b890 4881ec88000000 sub rsp, 88h
fffff802`7100b897 33c0 xor eax, eax
fffff802`7100b899 4889442478 mov qword ptr [rsp+78h], rax
fffff802`7100b89e c744247020000000 mov dword ptr [rsp+70h], 20h
fffff802`7100b8a5 89442468 mov dword ptr [rsp+68h], eax
fffff802`7100b8aa 4889442460 mov qword ptr [rsp+60h], rax
fffff802`7100b8af 89442458 mov dword ptr [rsp+58h], eax
fffff802`7100b8b3 8b8424e0000000 mov eax, dword ptr [rsp+0E0h]
fffff802`7100b8ba 89442450 mov dword ptr [rsp+50h], eax
fffff802`7100b8be 488b8424d8000000 mov rax, qword ptr [rsp+0D8h]
fffff802`7100b8c5 4889442448 mov qword ptr [rsp+48h], rax
fffff802`7100b8cb 8b8424d0000000 mov eax, dword ptr [rsp+0D0h]
fffff802`7100b8d2 89442440 mov dword ptr [rsp+40h], eax
fffff802`7100b8d6 8b8424c8000000 mov eax, dword ptr [rsp+0C8h]
fffff802`7100b8dd 89442438 mov dword ptr [rsp+38h], eax
fffff802`7100b8e1 8b8424c0000000 mov eax, dword ptr [rsp+0C0h]
fffff802`7100b8e8 89442430 mov dword ptr [rsp+30h], eax
fffff802`7100b8ec 8b8424b8000000 mov eax, dword ptr [rsp+0B8h]

```

nt!NtCreateFile+16

nt!NtCreateFile+16 giúp nhìn thẳng vào phần xử lý chính của hàm, bỏ qua đoạn khởi tạo stack, giúp quan sát logic trọng tâm dễ hơn

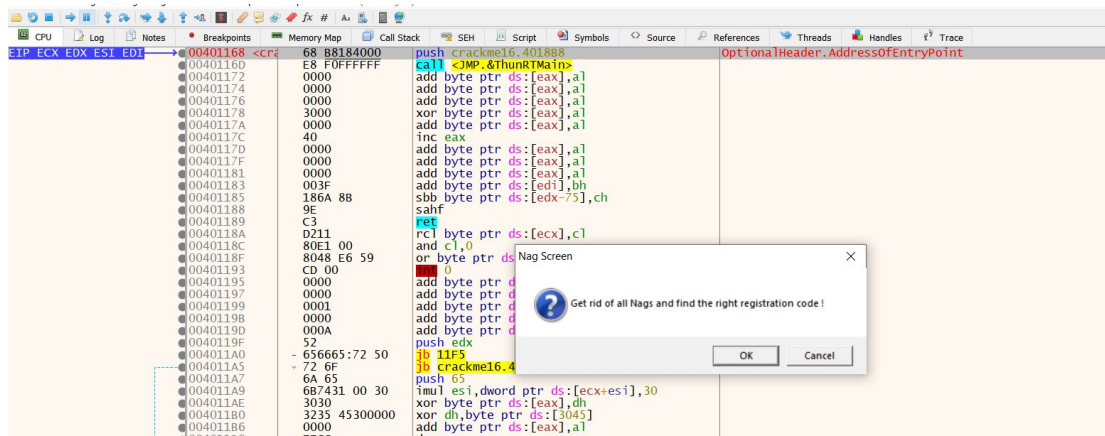
```

Disassembly - Dump C:\Windows\livekd.dmp - WinDbg:10.0.19041.5609 AMD64
Offset: nt!NtCreateFile+16
fffff802`7100b882 cc int 3
fffff802`7100b883 cc int 3
fffff802`7100b884 cc int 3
fffff802`7100b885 cc int 3
fffff802`7100b886 cc int 3
fffff802`7100b887 cc int 3
fffff802`7100b888 cc int 3
fffff802`7100b889 cc int 3
fffff802`7100b88a cc int 3
fffff802`7100b88b cc int 3
fffff802`7100b88c cc int 3
fffff802`7100b88d cc int 3
fffff802`7100b88e cc int 3
fffff802`7100b88f cc int 3
nt!NtCreateFile:
fffff802`7100b890 4881ec88000000 sub rsp, 88h
fffff802`7100b897 33c0 xor eax, eax
fffff802`7100b899 4889442478 mov qword ptr [rsp+78h], rax
fffff802`7100b89e c744247020000000 mov dword ptr [rsp+70h], 20h
fffff802`7100b8a5 89442468 mov dword ptr [rsp+68h], eax
fffff802`7100b8aa 4889442460 mov qword ptr [rsp+60h], rax
fffff802`7100b8b3 8b8424e0000000 mov eax, dword ptr [rsp+0E0h]
fffff802`7100b8ba 89442450 mov dword ptr [rsp+50h], eax
fffff802`7100b8be 488b8424d8000000 mov rax, qword ptr [rsp+0D8h]
fffff802`7100b8c5 4889442448 mov qword ptr [rsp+48h], rax
fffff802`7100b8cb 8b8424d0000000 mov eax, dword ptr [rsp+0D0h]
fffff802`7100b8d2 89442440 mov dword ptr [rsp+40h], eax
fffff802`7100b8d6 8b8424c8000000 mov eax, dword ptr [rsp+0C8h]
fffff802`7100b8dd 89442438 mov dword ptr [rsp+38h], eax
fffff802`7100b8e1 8b8424c0000000 mov eax, dword ptr [rsp+0C0h]
fffff802`7100b8e8 89442430 mov dword ptr [rsp+30h], eax
fffff802`7100b8ec 8b8424b8000000 mov eax, dword ptr [rsp+0B8h]
fffff802`7100b8f3 89442428 mov dword ptr [rsp+28h], eax
fffff802`7100b8f7 488b8424b0000000 mov rax, qword ptr [rsp+0B0h]
fffff802`7100b8ff 4889442420 mov qword ptr [rsp+20h], rax
fffff802`7100b904 e817000000 call nt!IopCreateFile (fffff802`7100b920)
fffff802`7100b909 4881c488000000 add rsp, 88h

```

Bài tập Crackme16

Bước 1: Em bấm Run để chạy file. Ngay khi cửa sổ quảng cáo hoặc thông báo đầu tiên hiện ra, em dừng chương trình ngay tại đó



Bước 2: em xem Call Stack để xác định chính xác hàm hoặc địa chỉ đã kích hoạt cửa sổ quảng cáo. Khi tìm thấy, em click để di chuyển đến lệnh gọi đó.

Thread ID	Address	To	From	Size	Party	Comment
6980 - Main Thread	0019F58C	764D4A14	775410CC	30	System	win32u.NtUserWaitMessage+C
	0019F58C	76531F28	764D4A14	CC	System	user32.DialogBoxIndirectParamAorW+114
	0019F688	76530CC1	76531F28	15C	System	user32.SoftwareModalMessageBox+728
	0019F7E4	76531428	76530CC1	90	System	user32.DrawStateA+1421
	0019F874	0F0AE119	76531428	38	User	user32.MessageBoxIndirectA+E8
	0019F8AC	0F0AEC81	0F0AE119	24	User	msvbvm50.__vbaVarDateVar+4078D
	0019F8D0	0F0AEFAF	0F0AEC81	30	User	msvbvm50.__vbaVarDateVar+405F5
	0019F900	0F0A6394	0F0AEFAF	6C	User	msvbvm50.__vbaVarDateVar+40923
	0019F96C	0F0D414D	0F0A6394	74	User	msvbvm50.__vbaVarDateVar+44D08
	0019F9E0	00402D03	0F0D414D	D4	User	msvbvm50.rtcMsgBox+F3
	0019FAB4	0F01E5A9	00402D03	14	User	crackme16.00402D03
	0019FAC8	0F001AA3	0F01E5A9	8	User	msvbvm50.__vbaErase+2A0
	0019FAD0	0F01E583	0F001AA3	1C	User	msvbvm50.0F001AA3
	0019FAEC	0F02B781	0F01E583	3C	User	msvbvm50.__vbaErase+27A
	0019FB28	0F01F8C2	0F02B781	E0	User	msvbvm50.__vbaHResultCheckObj+296
	0019FC08	0F01FA4A	0F01F8C2	74	User	msvbvm50.__vbaFreeStrList+21A
	0019FC7C	0F029A57	0F01FA4A	24	User	msvbvm50.__vbaFreeStrList+A2
	0019FCA0	0F0299E9	0F029A57	20	User	msvbvm50.__vbaStrVarMove+3786
	0019FCC0	0F05860C	0F0299E9	38	User	msvbvm50.__vbaStrVarMove+3748
	0019FCF8	0F0584EF	0F05860C	44	User	msvbvm50.TipGetAddressOfPreddeclaredInstance+BC
	0019FD3C	0F0120C4	0F0584EF	154	User	msvbvm50.EbResetProjectNormal+246
	0019FE90	0F00FF69	0F0120C4	1C	User	msvbvm50.EbLoadRunTime+DF3
	0019FEAC	0F00A583	0F00FF69	260158	User	msvbvm50.EbSetContextWorkerThread+1040
	00400004	00000000	0F00A583		User	msvbvm50.ThunRTMain+3C4
792	0077FF74	768AFCC9	77C548FC	10	System	ntdll.ZwWaitForWorkViaWorkerFactory+C
	0077FF84	77C47C6E	768AFCC9	5C	System	kernel32.BaseThreadInitThunk+19
	0077FFE0	77C47C3E	77C47C6E	10	System	ntdll.RtlGetAppContainerNamedObjectPath+11E
	0077FFD0	00000000	77C47C3E		User	ntdll.RtlGetAppContainerNamedObjectPath+EE
556	0087FF74	768AFCC9	77C548FC	10	System	ntdll.ZwWaitForWorkViaWorkerFactory+C
	0087FF84	77C47C6E	768AFCC9	5C	System	kernel32.BaseThreadInitThunk+19
	0087FFD0	77C47C3E	77C47C6E	10	System	ntdll.RtlGetAppContainerNamedObjectPath+11E

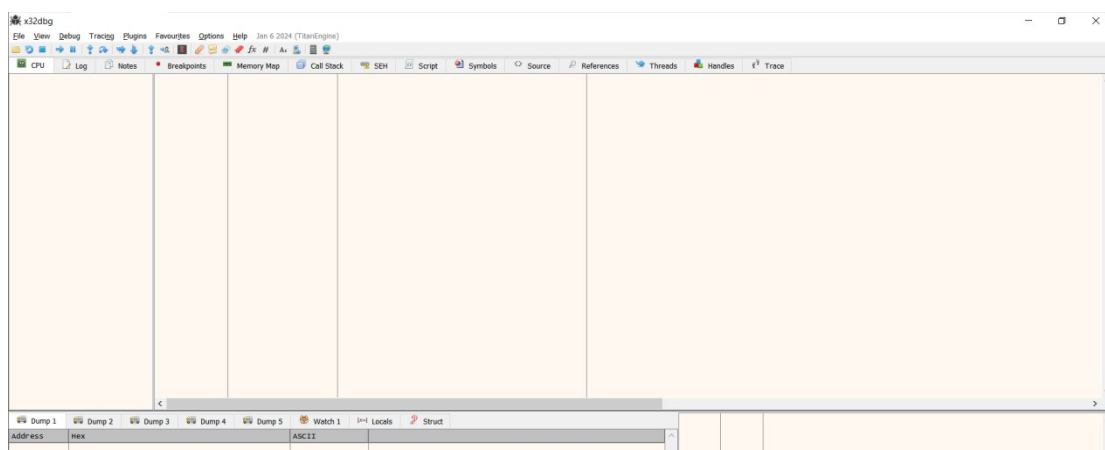
Bước 3: Tại cửa sổ Call Stack, em tìm thấy lệnh call chính là nguyên nhân gọi ra nag. Em tiến hành cài breakpoint ngay tại lệnh đó

00402CE8	8D45 9C	lea eax,dword ptr ss:[ebp-64]	
00402CEB	50	push eax	
00402CEC	8D45 CC	lea eax,dword ptr ss:[ebp-34]	
00402CEF	50	push eax	
00402CF0	8D45 DC	lea eax,dword ptr ss:[ebp-24]	
00402CF3	50	push eax	
00402CF4	E8 21E4FFFF	call <JMP.&__vbaI4Var>	
00402CF9	50	push eax	
00402CFA	8D45 AC	lea eax,dword ptr ss:[ebp-54]	
00402CFD	50	push eax	
00402CFE	E8 1DE4FFFF	call <JMP.&rtcMsgBox>	
00402D03	8D95 5CFFFFFF	lea edx,dword ptr ss:[ebp-A4]	
00402D09	8D4D BC	lea ecx,dword ptr ss:[ebp-44]	
00402D0C	8985 64FFFFFF	mov dword ptr ss:[ebp-9C],eax	
00402D12	89BD 5CFFFFFF	mov dword ptr ss:[ebp-A4],edi	
00402D18	E8 09E4FFFF	call <JMP.&__vbaVarMove>	
00402D1D	8D45 8C	lea eax,dword ptr ss:[ebp-74]	
00402D20	50	push eax	
00402D21	8D45 9C	lea eax,dword ptr ss:[ebp-64]	
00402D24	50	push eax	
00402D25	6A 02	push 2	
00402D27	E8 E8E3FFFF	call <JMP.&__vbaFreeVarList>	
00402D2C	83C4 0C	add esp,C	

Bước 4 em khởi động lại và chạy thử. Breakpoint giúp chặn không cho nag hiển thị. Em patch lệnh call thành nop để từ giờ không còn nag xuất hiện nữa

00402CFA	8D45 AC	lea eax,dword ptr ss:[ebp-54]	
00402CFD	50	push eax	
00402CFE	90	nop	
00402CFF	90	nop	
00402D00	90	nop	
00402D01	90	nop	
00402D02	90	nop	
00402D03	8D95 5CFFFFFF	lea edx,dword ptr ss:[ebp-A4]	
00402D09	8D4D BC	lea ecx,dword ptr ss:[ebp-44]	ecx: __vbaI4Var+C
00402D0C	8985 64FFFFFF	mov dword ptr ss:[ebp-9C],eax	
00402D12	89BD 5CFFFFFF	mov dword ptr ss:[ebp-A4],edi	
00402D18	E8 09E4FFFF	call <JMP.&__vbaVarMove>	
00402D1D	8D45 8C	lea eax,dword ptr ss:[ebp-74]	
00402D20	50	push eax	
00402D21	8D45 9C	lea eax,dword ptr ss:[ebp-64]	
00402D24	50	push eax	
00402D25	6A 02	push 2	
00402D27	E8 E8E3FFFF	call <JMP.&__vbaFreeVarList>	
00402D2C	83C4 0C	add esp,C	
00402D2F	8D45 BC	lea eax,dword ptr ss:[ebp-44]	
00402D32	C745 84 01000000	mov dword ptr ss:[ebp-7C],1	[ebp-7C]:L"Nag Screen "

Bước 5: Sau khi patch xong, em chạy lại để xem thành quả. Nhưng không ngờ debugger tự động tắt và thoát, cho thấy có lỗi ở đâu đó trong code em vừa sửa



Bước 6: Vì chương trình đóng đột ngột mà không hiển thị giao diện, em bật Trace Coverage để dò từng lệnh đã chạy. Nhờ đó, em thấy được dòng lệnh cuối cùng trước khi chương trình bị đóng, như trong ảnh chụp

00402CFA	8D45 AC	lea eax,dword ptr ss:[ebp-54]	
00402CFD	50	push eax	
00402CFE	E8 1DE4FFFF	call <JMP.&rtcMsgBox>	
00402D03	8D95 5CFFFFFF	lea edx,dword ptr ss:[ebp-A4]	
00402D09	8D4D BC	lea ecx,dword ptr ss:[ebp-44]	ecx: __vbaI4Var+C
00402D0C	8985 64FFFFFF	mov dword ptr ss:[ebp-9C],eax	
00402D12	898D 5CFFFFFF	mov dword ptr ss:[ebp-A4],edi	
00402D18	E8 09E4FFFF	call <JMP.&_vbaVarMove>	
00402D1D	8D45 8C	lea eax,dword ptr ss:[ebp-74]	
00402D20	50	push eax	
00402D21	8D45 9C	lea eax,dword ptr ss:[ebp-64]	
00402D24	50	push eax	
00402D25	6A 02	push 2	
00402D27	E8 E8E3FFFF	call <JMP.&_vbaFreeVarList>	
00402D2C	83C4 0C	add esp,C	
00402D2F	8D45 BC	lea eax,dword ptr ss:[ebp-44]	
00402D32	C745 84 01000000	mov dword ptr ss:[ebp-7C],1	[ebp-7C]:L"Nag Screen "
00402D39	C785 7CFFFFFF 0	mov dword ptr ss:[ebp-84],8003	
00402D43	50	push eax	
00402D44	8D85 7CFFFFFF	lea eax,dword ptr ss:[ebp-84]	
00402D4A	50	push eax	
00402D4B	E8 BEE3FFFF	call <JMP.&_vbaVarTstEq>	
00402D50	66:85C0	test ax,ax	
00402D53	75 05	jne crackme16.402D5A	
00402D55	E8 AEE3FFFF	call <JMP.&_vbaEnd>	
00402D5A	8975 FC	mov dword ptr ss:[ebp-4],esi	
00402D5D	68 982D4000	push crackme16.402D98	
00402D62	EB 13	jmp crackme16.402D77	

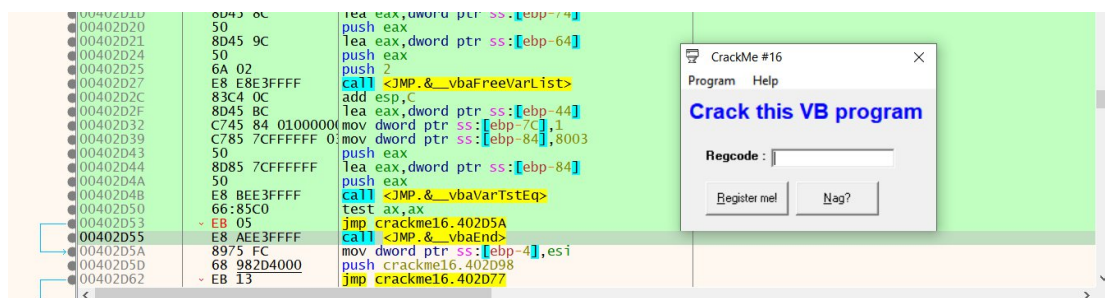
Bước 7: Khi chạy đến 00402D55, chương trình thoát. Em sẽ vô hiệu hóa dòng lệnh này. May mắn là ngay phía trên có một lệnh jne – nếu thay đổi điều kiện, em có thể nhảy qua khỏi dòng gây lỗi

00402D50	66:85C0	test ax,ax	
00402D53	75 05	jne crackme16.402D5A	
00402D55	E8 AEE3FFFF	call <JMP.&_vbaEnd>	
00402D5A	8975 FC	mov dword ptr ss:[ebp-4],esi	

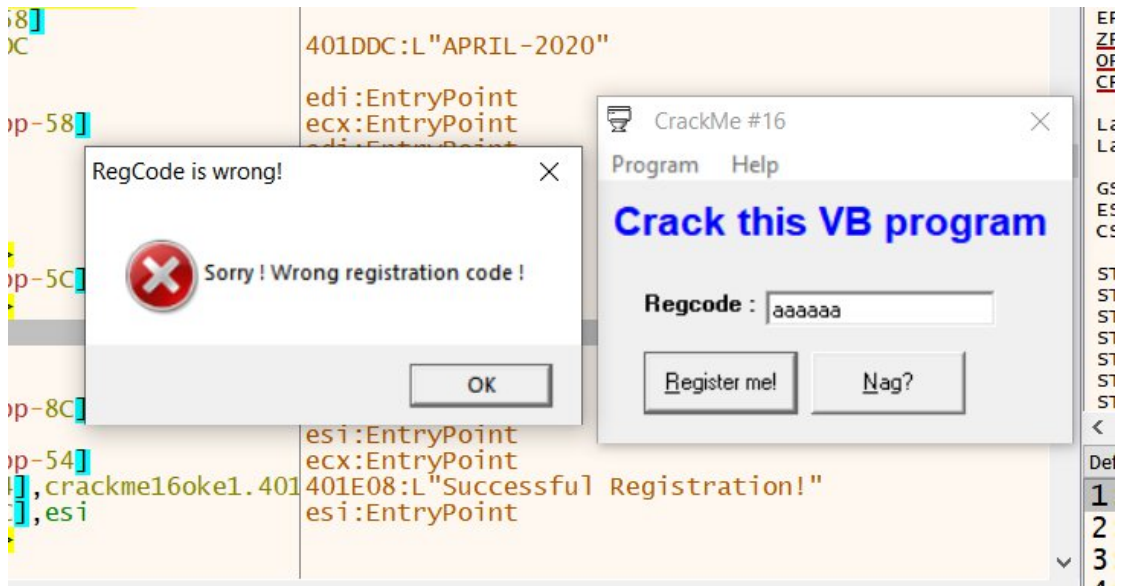
Bước 8: Em sửa lệnh jne thành jmp để nó luôn nhảy bất kể điều kiện. Nhờ vậy, lệnh call hoặc jmp ở dưới gây đóng chương trình sẽ bị bỏ qua hoàn toàn

00402D50	66:85C0	test ax,ax	
00402D53	EB 05	jmp crackme16.402D5A	
00402D55	E8 AEE3FFFF	call <JMP.&_vbaEnd>	
00402D5A	8975 FC	mov dword ptr ss:[ebp-4],esi	
00402D5D	68 982D4000	push crackme16.402D98	
00402D62	EB 13	jmp crackme16.402D77	

Em đã qua được thông báo quảng cáo ban đầu – không còn xuất hiện nữa.



Bước 9: Tiếp theo, em sẽ crack Regcode. Chương trình có cơ chế kiểm tra và hiển thị lỗi nếu nhập sai, như trong ảnh bên dưới



Em suy luận:

Thay vì dùng String References như các bài trước, em sẽ tìm Regcode bằng cách phân tích luồng xử lý khi bấm nút Register me

Em dùng công cụ VB Decompiler Lite để xác định địa chỉ code gắn với sự kiện Click của nút đó. Trong công cụ, nút này có tên Command1. Em xem mã nguồn bên phải, tìm địa chỉ bắt đầu xử lý và copy vào x64dbg

Tại đó, em đặt breakpoint rồi chạy từng bước (step) để theo dõi cách chương trình so sánh Regcode. Nhờ vậy, em sẽ tìm ra cơ chế kiểm tra và xác định được Regcode chính xác

Type	Address	Module/Label/Exception	State	Disassembly
Software	00402814	crackme16oke1.exe	Enabled	push ebp

và

Address	Disassembly	Comment
00402814	push ebp	
8BEC	mov ebp,esp	
83EC 0C	sub esp,c	
68 66104000	push <JMP.&_vbaExceptionHandler>	
64:A1 00000000	mov eax,dword ptr [0]	
50	push eax	
64:8925 00000000	mov dword ptr [0],esp	
81EC B0000000	sub esp,80	
53	push ebx	
8B5D 08	mov ebx,dword ptr ss:[ebp+8]	
8BC3	mov eax,ebx	
56	push esi	
83E3 FE	and ebx,FFFFFFFF	
57	push edi	
8965 F4	mov dword ptr ss:[ebp-C],esp	[ebp-C]:FrameRect+132
83E0 01	and eax,1	
8B3B	mov edi,dword ptr ds:[ebx]	
C745 F8 00104000	mov dword ptr ss:[ebp-8],crackme16oke1.401C	[ebp-8]:_vbaErase+2A0
53	push ebx	
8945 FC	mov dword ptr ss:[ebp-4],eax	[ebp-4]:"B0"
895D 08	mov dword ptr ss:[ebp+8],ebx	
FE57 04	call dword ptr ds:[edi+4]	[edi+304]:GetSystemMetricsForDpi+1747
8BBF 04030000	mov edi,dword ptr ds:[edi+304]	
33F6	xor esi,esi	
53	push ebx	
8975 DC	mov dword ptr ss:[ebp-24],esi	
8975 CC	mov dword ptr ss:[ebp-34],esi	
8975 BC	mov dword ptr ss:[ebp-44],esi	
8975 AC	mov dword ptr ss:[ebp-54],esi	
8975 A8	mov dword ptr ss:[ebp-58],esi	
8975 A4	mov dword ptr ss:[ebp-5C],esi	
8975 94	mov dword ptr ss:[ebp-6C],esi	
8975 84	mov dword ptr ss:[ebp-7C],esi	[ebp-7C]:PolyPatBlt+11F

Bước 10: Em tiếp tục chạy từng bước và phát hiện một lệnh je. Lệnh này kiểm tra điều kiện và nếu đúng, nó sẽ nhảy qua phần hiển thị thông báo đăng ký thành công

004028D1	F7DF	neg edi	
004028D3	E8 60E8FFFF	call <JMP.&_vbaFreeStr>	
004028D8	8D4D A4	lea ecx,dword ptr ss:[ebp-5C]	
004028DB	E8 52E8FFFF	call <JMP.&_vbaFreeObj>	
004028E0	66:3BFE	cmp di,si	
004028E3	0F84 F3000000	je crackme16oke1.4029DC	
004028E9	6A 08	push 8	
004028EB	8D95 74FFFFFF	lea edx,dword ptr ss:[ebp-8C]	
004028F1	5E	pop esi	
004028F2	8D4D AC	lea ecx,dword ptr ss:[ebp-54]	
004028F5	C785 7CFFFFFF	mov dword ptr ss:[ebp-84],crackme16oke1.401E08:L"Successful Registration!"	
004028FF	89B5 74FFFFFF	mov dword ptr ss:[ebp-8C],esi	
00402905	E8 22E8FFFF	call <JMP.&_vbaVarCopy>	
0040290A	6A 03	push 3	
0040290C	8D95 74FFFFFF	lea edx,dword ptr ss:[ebp-8C]	
00402912	5B	pop ebx	
00402913	8D4D DC	lea ecx,dword ptr ss:[ebp-24]	
00402916	C785 7CFFFFFF	mov dword ptr ss:[ebp-84],31:'1'	
00402920	899D 74FFFFFF	mov dword ptr ss:[ebp-8C],ebx	
00402926	E8 FB77FFFF	call <JMP.&_vbaVarMove>	
0040292B	8D95 74FFFFFF	lea edx,dword ptr ss:[ebp-8C]	
00402931	8D4D CC	lea ecx,dword ptr ss:[ebp-34]	
00402934	C785 7CFFFFFF	mov dword ptr ss:[ebp-84],crackme16oke1.401E50:L"Congratz !!!!!"	
0040293E	89B5 74FFFFFF	mov dword ptr ss:[ebp-8C],esi	
00402944	E8 E377FFFF	call <JMP.&_vbaVarCopy>	
00402949	6A 0A	push A	
0040294B	8D45 84	lea eax,dword ptr ss:[ebp-7C]	
0040294E	5E	pop esi	
0040294F	BF 04000280	mov edi,80020004	
00402954	50	push eax	
00402955	8D45 94	lea eax,dword ptr ss:[ebp-6C]	

Bước 11: Em quay lại phía trên lệnh je một chút để tìm hiểu điều kiện. Em phát hiện một hàm so sánh chuỗi, có vẻ như nó đang so sánh key mình nhập với một giá trị cố định nào đó

004028B4	50	push eax	
004028B5	E8 84E8FFFF	call <JMP.&_vbaHresultCheckObj>	
004028BD	FF75 A8	push dword ptr ss:[ebp-58]	[ebp-58]:L"
004028C2	E8 83E8FFFF	call <JMP.&_vbaStrCmp>	401DDC:L"Al
004028C7	8BF8	mov edi,eax	
004028C9	8D4D A8	lea ecx,dword ptr ss:[ebp-58]	[ebp-58]:L"
004028CC	F7DF	neg edi	
004028CE	1BFF	sbb edi,edi	
004028D0	47	inc edi	
004028D1	F7DF	neg edi	
004028D3	E8 60E8FFFF	call <JMP.&_vbaFreeStr>	
004028D8	8D4D A4	lea ecx,dword ptr ss:[ebp-5C]	
004028DB	E8 52E8FFFF	call <JMP.&_vbaFreeObj>	
004028E0	66:3BFE	cmp di,si	
004028E3	0F84 F3000000	je crackme16oke1.4029DC	
004028E9	6A 08	push 8	
004028EB	8D95 74FFFFFF	lea edx,dword ptr ss:[ebp-8C]	
004028F1	5E	pop esi	
004028F2	8D4D AC	lea ecx,dword ptr ss:[ebp-54]	
004028F5	C785 7CFFFFFF	mov dword ptr ss:[ebp-84],crackme16oke1.401E08:L"Successful Registration!"	
004028FF	89B5 74FFFFFF	mov dword ptr ss:[ebp-8C],esi	
00402905	E8 22E8FFFF	call <JMP.&_vbaVarCopy>	
0040290A	6A 03	push 3	
0040290C	8D95 74FFFFFF	lea edx,dword ptr ss:[ebp-8C]	

đây là Regcode chính xác – chuỗi mà chương trình đang so sánh với dữ liệu nhập vào. Em sẽ thử nhập nó để kiểm tra xem có thành công không.

DONE!



